Foodly

Advanced JavaScript and DOM Manipulation

Team name: KN (Foodly)
Members: Kakhar Nassyrkhan and Nurasyl Raiskaliev
Group: SE-2506

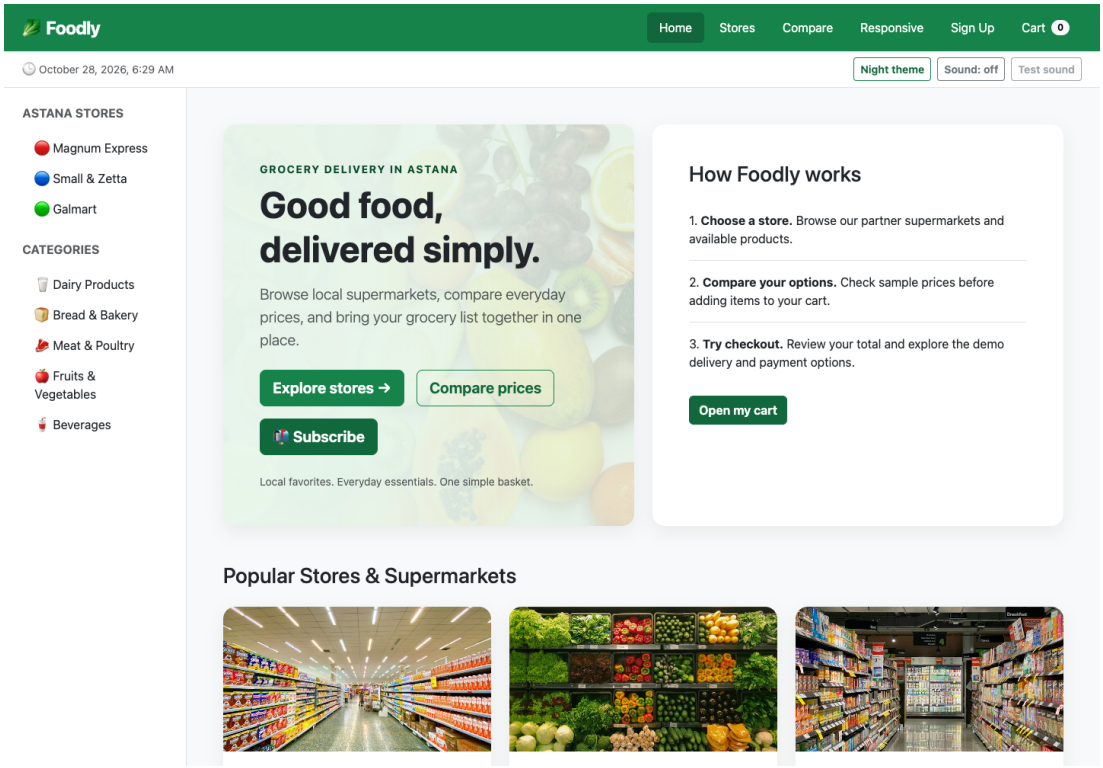


Figure 1. The updated Foodly homepage, captured from the working implementation.

Objective

Extend the existing food-delivery project with interactive JavaScript, DOM updates, keyboard and button events, callbacks, objects, arrays, sound and animations. Preserve a clear shopping flow and correct the existing form-validation problems.

Project links

Deployed website:
https://vuus1d.github.io/Food-Delivery-Platform-KN/
Shared repository:
https://github.com/vuus1d/Food-Delivery-Platform-KN

This is an educational frontend. Prices and orders are examples; no real account, email, payment or delivery is created. Screenshots use synthetic test data.

Requirements and chosen options

The supplied Assignment 5 PDF permits alternatives. The implementation below selects concrete options and maps every required area to source files and report evidence.

Requirement	Selected implementation	Page
DOM selection and content	Rating controls, catalog and live messages.	3
Dynamic styling	Persistent day/night theme.	4
Attribute manipulation	Read More: display, hidden and aria-expanded.	3
Button and keyboard events	Click listeners; arrow-key menu navigation.	3-5
Callbacks	Three-step checkout; move(direction, onChange).	6
Switch statements	Catalog sorting, payment labels, keyboard keys.	5, 7-8
Objects, arrays, loops	checkout methods; product arrays; review loop.	8
Higher-order functions	map, filter, reduce and forEach callbacks.	7-8
Sound	Web Audio notification with overlap protection.	9
Animations	Rating pulse, popup/toast entry; reduced motion.	10
Structure and readability	External HTML/CSS/JS, named reusable functions.	12
Theme and UX	Grocery-specific interactions, mobile flow and focus.	3-12
Validation correction	Shared checks, inline errors, reset and success.	11
Deployment	Commit, push, GitHub Pages build and live check.	14
Report and defense	Code/result screenshots, steps, reflection and guide.	All; 15

The callback requirement is satisfied by the multi-step form alternative. A real asynchronous contact POST is not claimed. The original FAQ, carousel and live clock are retained as supporting features.

1. DOM updates and attributes

1. Select all rating buttons with querySelectorAll and locate the status/advice elements by ID.
2. On click, update star classes and aria-pressed; write the selected rating using textContent.
3. Read More changes style.display, hidden, the button label and aria-expanded together.

js/interactions.js · lines 21-43

```
21  const moreButton = document.getElementById("read-more");
22  moreButton?.addEventListener("click", () => {
23      const content = document.getElementById("shopping-details");
24      const expanded = moreButton.getAttribute("aria-expanded") !== "true";
25      content.style.display = expanded ? "block" : "none";
26      content.hidden = !expanded;
27      moreButton.setAttribute("aria-expanded", String(expanded));
28      moreButton.textContent = expanded ? "Read less" : "Read more";
29  });
30
31  const stars = document.querySelectorAll(".rating-star");
32  stars.forEach(star => star.addEventListener("click", () => {
33      const rating = Number(star.dataset.rating);
34      stars.forEach(button => {
35          button.classList.toggle("selected", Number(button.dataset.rating) <= rating);
36          button.setAttribute("aria-pressed", String(button === star));
37      });
38      document.getElementById("rating-status").textContent = `You rated Foodly ${rating} out
    of 5. Thank you! (This visit only.)`;
39      const card = document.getElementById("experience-card");
40      card.classList.remove("celebrate");
41      void card.offsetWidth; // Restart the animation when another rating is selected.
42      card.classList.add("celebrate");
43  }));
```

Figure 2. Actual source: js/interactions.js, DOM selection, content and attribute updates.

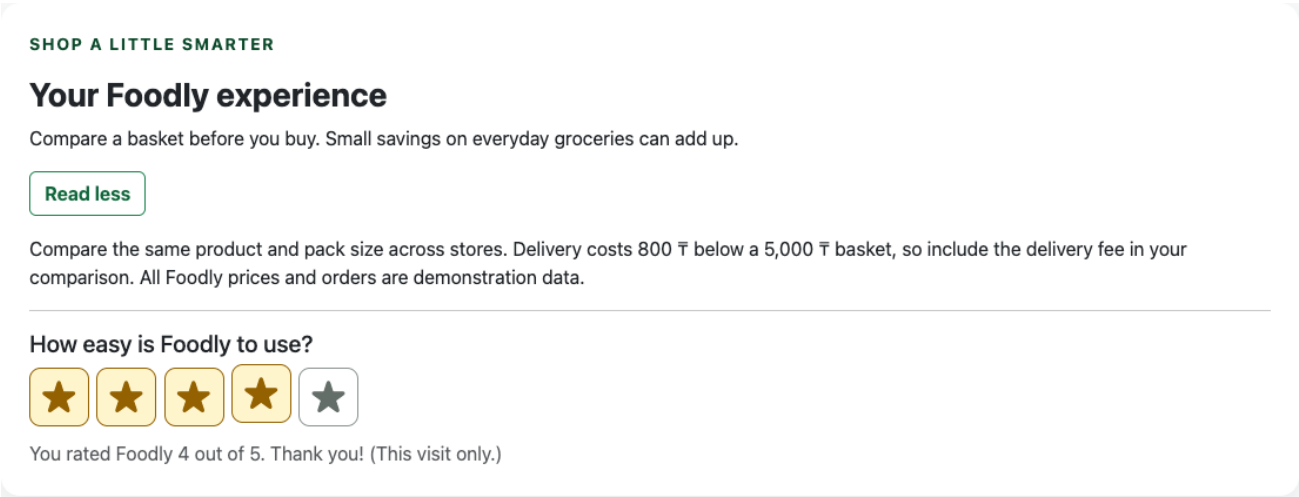


Figure 3. Four stars selected and shopping advice expanded, without a page reload.

The selected rating is temporary for this visit. The advice explains the demo delivery fee and remains within the grocery-shopping theme.

2. Day and night styling

1. Read the saved preference with a safe light-theme fallback if storage is unavailable.
2. Set data-bs-theme on the HTML root; CSS changes surfaces, text, borders and buttons.
3. Update the toggle label and pressed state, then save the preference for the next visit.

js/theme.js · lines 5-17

```
5  function applyTheme(theme) {
6      document.documentElement.setAttribute("data-bs-theme", theme);
7      button.setAttribute("aria-pressed", String(theme === "dark"));
8      button.textContent = theme === "dark" ? "Day theme" : "Night theme";
9  }
10 let savedTheme = "light";
11 try { savedTheme = localStorage.getItem("foodly-theme") || "light"; } catch { /* use default
   */ }
12 applyTheme(savedTheme === "dark" ? "dark" : "light");
13 button.addEventListener("click", () => {
14     const theme = document.documentElement.dataset.bsTheme === "dark" ? "light" : "dark";
15     applyTheme(theme);
16     try { localStorage.setItem("foodly-theme", theme); } catch { /* still works on this page
   */ }
17 });
```

Figure 4. Actual source: js/theme.js, theme application and click listener.

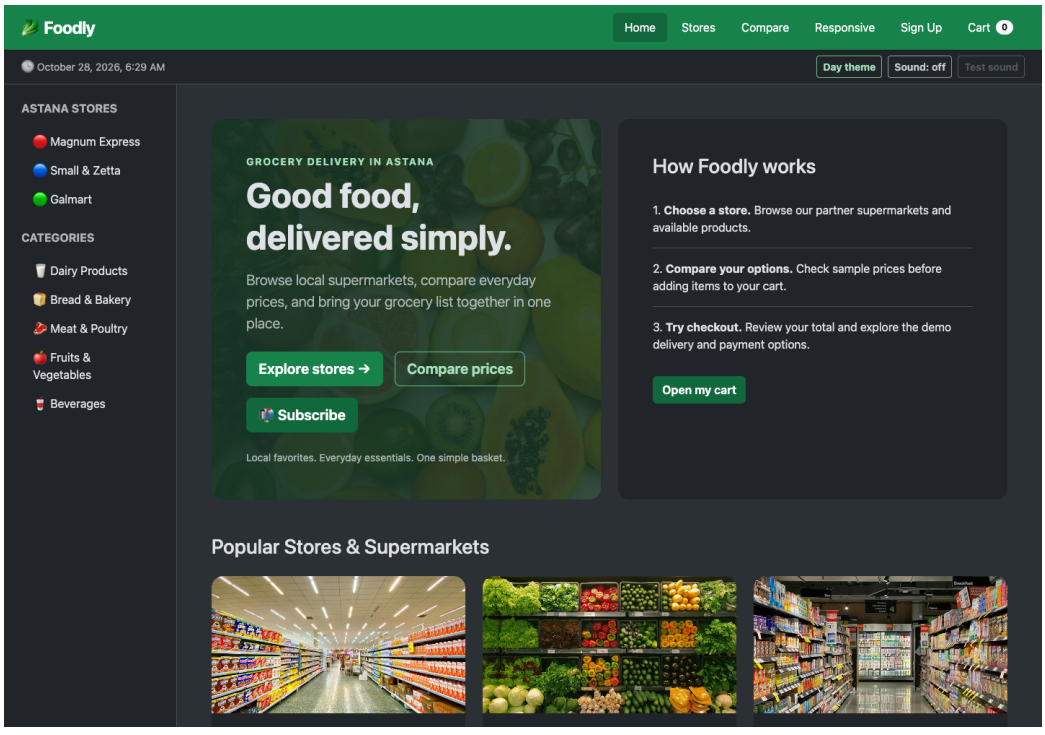


Figure 5. The homepage in night mode; the navigation, hero, cards and sidebar remain legible.

Bootstrap pages share this toolbar. The previous responsive.html exercise intentionally keeps its standalone light-only CSS. Dark secondary-button colors were adjusted after an automated contrast check.

3. Button and keyboard events

1. Use addEventListener for custom buttons; no inline onclick attributes are used.

2. Listen for keydown on the main navigation and identify the focused visible link.

3. A switch maps arrows, Home and End to a new index. Wrap at the ends and call focus(). Leave normal Tab navigation unchanged.

js/interactions.js · lines 3-19

```
3  document.querySelectorAll('nav[aria-label="Main navigation"]').forEach(nav => {
4      nav.addEventListener("keydown", event => {
5          const links = [...nav.querySelectorAll("li a")].filter(link =>
   link.getClientRects().length);
6          const current = links.indexOf(document.activeElement);
7          if (current < 0) return;
8          let next = current;
9          switch (event.key) {
10             case "ArrowRight": case "ArrowDown": next = (current + 1) % links.length; break;
11             case "ArrowLeft": case "ArrowUp": next = (current - 1 + links.length) %
   links.length; break;
12             case "Home": next = 0; break;
13             case "End": next = links.length - 1; break;
14             default: return;
15         }
16         event.preventDefault();
17         links[next].focus();
18     });
19 });
```

Figure 6. Actual source: js/interactions.js, keyboard navigation with switch and wraparound.



Figure 7. Focus moved to the final navigation item using ArrowLeft from Home.

Additional keyboard behavior

The subscription dialog traps Tab, closes with Escape and restores focus to its opener. Background content becomes inert while it is open. Checkout uses Bootstrap modal focus management. These behaviors support the same keyboard-first flow as the navigation.

4. Callbacks and multi-step form

1. Divide checkout into Delivery, Options and Review sections; keep their inputs in the DOM.

2. Create a checkout object with a current step. move(direction, onChange) validates Next and computes the next step.

3. Pass renderStep as a callback. It shows one section, updates progress and moves focus. Back keeps entered values.

4. Before completion, revalidate every step, require demo consent and show the computed total.

js/checkout.js · lines 27-33

```
27      // The caller supplies the rendering callback. Inputs remain in the DOM.
28      move(direction, onChange) {
29          if (direction > 0 && !validator.validate(steps[this.step])) return;
30          this.step = Math.max(0, Math.min(steps.length - 1, this.step + direction));
31          onChange(this.step);
32      }
33  };
```

Figure 8. Actual source: js/checkout.js. The callback is passed into move and invoked after the state change.

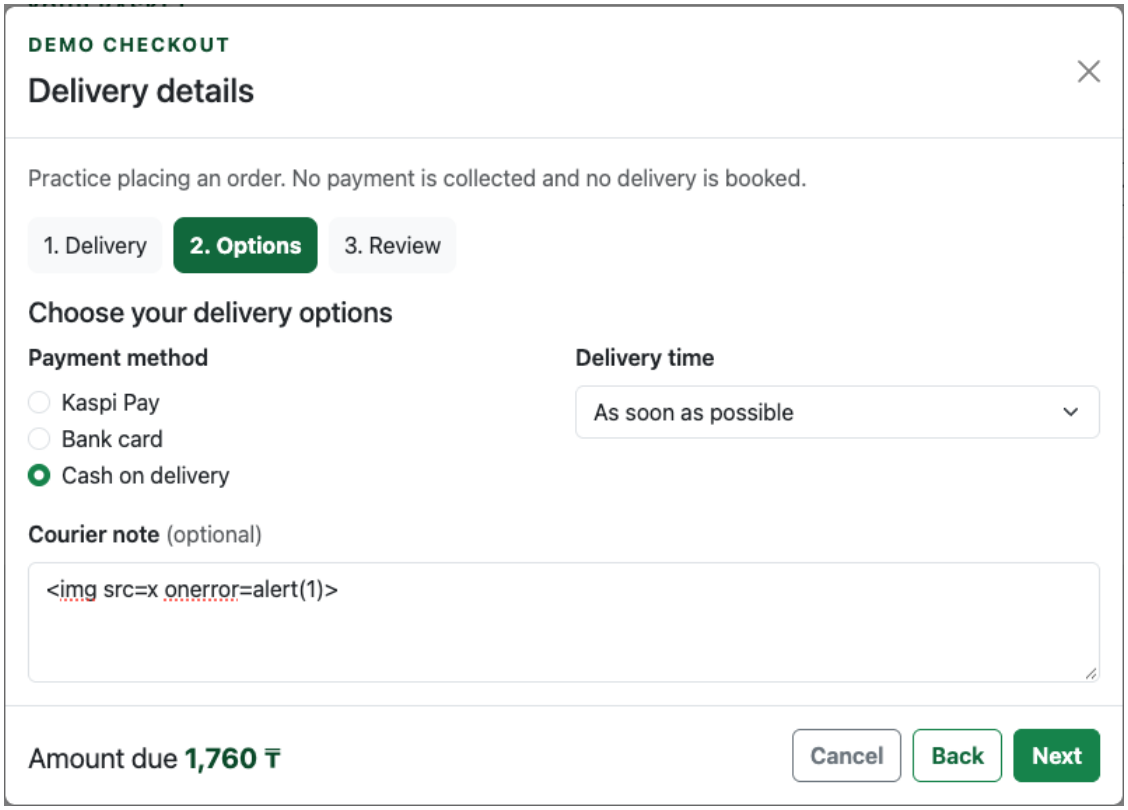


Figure 9. Step 2: payment, delivery window and optional courier note. Back and Next preserve input.

Next is blocked when the current step is invalid. Submitting from an earlier step advances only after validation; completion is allowed on Review.

5. Switch logic and catalog events

1. Create the selected store's product cards from its array of product objects.

2. Read the catalog search and sorting controls. Use a comparator selected by switch for price ascending, price descending, name or original order.

3. Filter visible cards, reorder existing DOM nodes and update the live product count. Show an empty state when nothing matches.

js/app.js · lines 168-176

```
168     const firstPrice = Number(first.querySelector(".add-product").dataset.price);
169     const secondPrice = Number(second.querySelector(".add-product").dataset.price);
170     switch (sort.value) {
171         case "price-asc": return firstPrice - secondPrice;
172         case "price-desc": return secondPrice - firstPrice;
173         case "name": return
    first.querySelector("h3").textContent.localeCompare(second.querySelector("h3").textContent);
174         default: return Number(first.dataset.order) - Number(second.dataset.order);
175     }
176 });
```

Figure 10. Actual source: js/app.js, catalog comparator selected by switch.

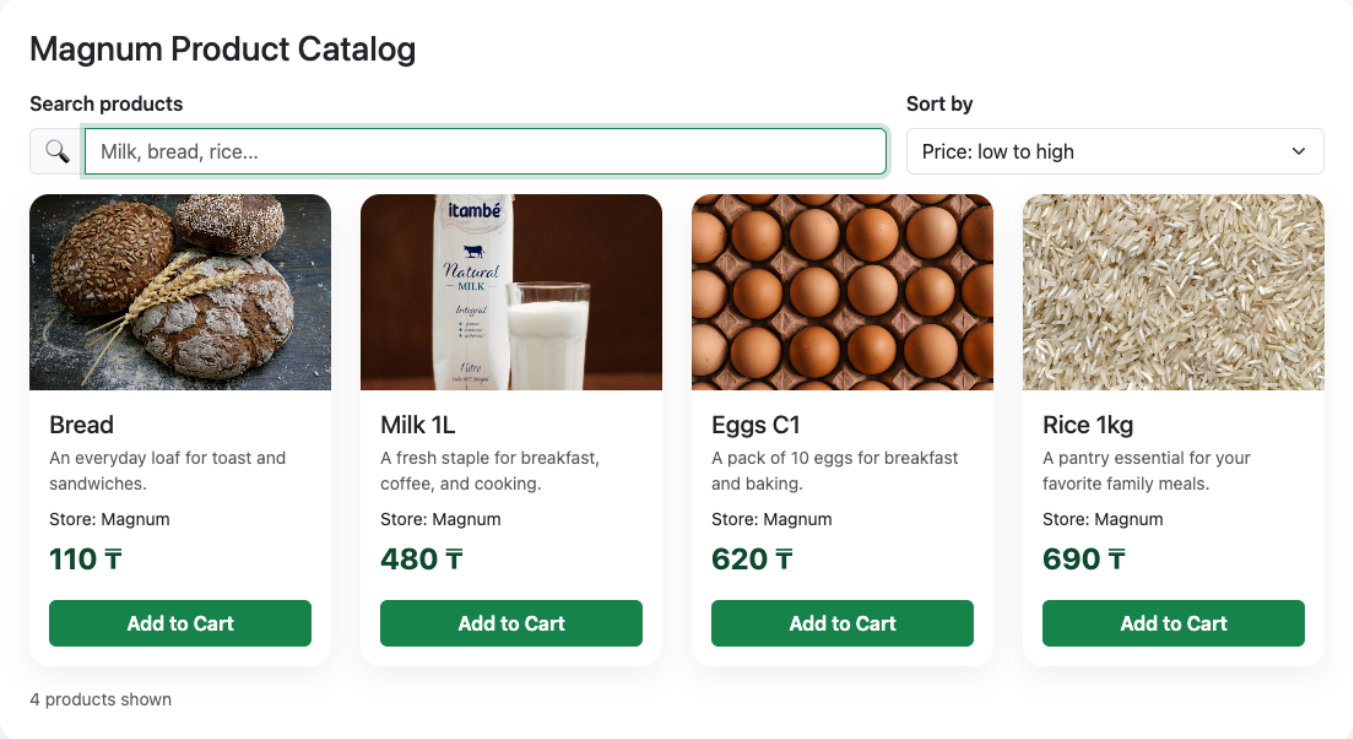


Figure 11. Magnum catalog sorted from low to high price. Search and Add to Cart work on the rendered cards.

Other switch statements map payment choices to readable review labels and keyboard keys to navigation actions. Comparison-page categories continue to filter their sample price rows.

6. Objects, arrays and functions

The checkout object holds state and methods. Product arrays describe grocery items. reduce computes totals, map creates review rows, filter selects fields/items and forEach updates groups of controls. These are higher-order functions because they accept callback functions.

js/checkout.js · lines 12-26

```
12  const checkout = {
13      step: 0,
14      totals() {
15          const subtotal = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);
16          const delivery = subtotal > 0 && subtotal < 5000 ? DELIVERY_FEE : 0;
17          return { subtotal, delivery, total: subtotal + delivery };
18      },
19      paymentLabel() {
20          switch (form.elements.payment.value) {
21              case "kaspi": return "Kaspi Pay (demo)";
22              case "card": return "Bank card (demo)";
23              case "cash": return "Cash on delivery (demo)";
24              default: return "Choose a payment method";
25          }
26      },
```

Figure 12. Actual source: js/checkout.js, object properties and methods, including totals and payment labels.

js/checkout.js · lines 49-57

```
49      review.replaceChildren();
50      for (const [label, value] of details) {
51          const term = document.createElement("dt");
52          const description = document.createElement("dd");
53          term.textContent = label;
54          description.textContent = value; // Personal input is never parsed as HTML.
55          review.append(term, description);
56      }
57  }
```

Figure 13. Actual source: an explicit for...of loop creates dt/dd pairs and uses textContent.

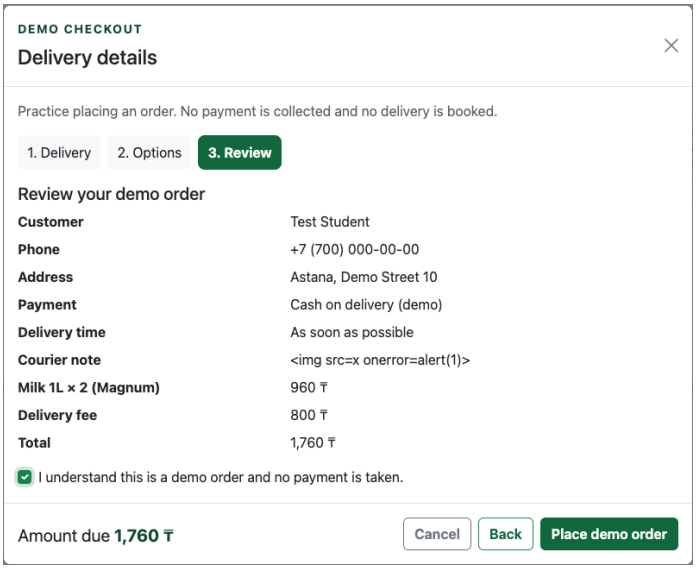


Figure 14. Review combines input and cart data. The HTML-like courier note is deliberately shown as plain text in this safety test.

7. Sound without overlap

1. Sound starts off. Enable it using Sound: on; Test sound and Add to Cart can then trigger a short notification.

2. Create a Web Audio oscillator and gain envelope after the user action. Change pitch from 660 Hz to 880 Hz and fade out.

3. Set the busy guard before awaiting the audio context. When playback ends, disconnect the nodes and release the guard.

js/interactions.js · lines 54-80

```
54  async function playNotification() {
55      if (!soundEnabled || busy) return;
56      busy = true; // Lock before awaiting resume, so rapid clicks cannot overlap.
57      try {
58          context ||= new AudioContextClass();
59          await context.resume();
60          if (!soundEnabled) { busy = false; return; }
61          const oscillator = context.createOscillator();
62          activeTone = oscillator;
63          const volume = context.createGain();
64          const now = context.currentTime;
65          oscillator.type = "sine";
66          oscillator.frequency.setValueAtTime(660, now);
67          oscillator.frequency.setValueAtTime(880, now + 0.09);
68          volume.gain.setValueAtTime(0, now);
69          volume.gain.linearRampToValueAtTime(0.12, now + 0.02);
70          volume.gain.exponentialRampToValueAtTime(0.001, now + 0.24);
71          oscillator.connect(volume);
72          volume.connect(context.destination);
73          oscillator.onended = () => {
74              oscillator.disconnect();
75              volume.disconnect();
76              busy = false;
77              activeTone = null;
78          };
79          oscillator.start(now);
80          oscillator.stop(now + 0.25);
```

Figure 15. Actual source: js/interactions.js, user-triggered audio, gain envelope and cleanup.



Figure 16. Toolbar with sound enabled and a separate test control.

Recorded check: two sounds started and ended; a burst of ten clicks never produced more than one active oscillator. Playback is verified through Web Audio events. Audible device volume was not measured.

8. Event-triggered animations

1. After a rating click, remove and re-add the celebration class so the card pulse can run again.

2. When a product is added, create a toast. CSS applies its entry animation; the notification disappears automatically or through its close button.

3. Honor prefers-reduced-motion by disabling animations and transitions. Keep the message and interaction available.

css/style.css · lines 222-226

```
222  .rating-star:hover { transform: translateY(-3px); }
223  .rating-star.selected { color: #936100; border-color: #936100; background: #fff5cd; }
224  .celebrate { animation: rating-pulse 360ms ease; }
225  @keyframes rating-pulse { 50% { transform: translateY(-4px); box-shadow: 0 12px 30px
     #16834b33; } }
226  .toast.show { animation: popup-in 180ms ease-out; }
```

Figure 17. Actual source: css/style.css, star feedback, card keyframes and toast entry animation.

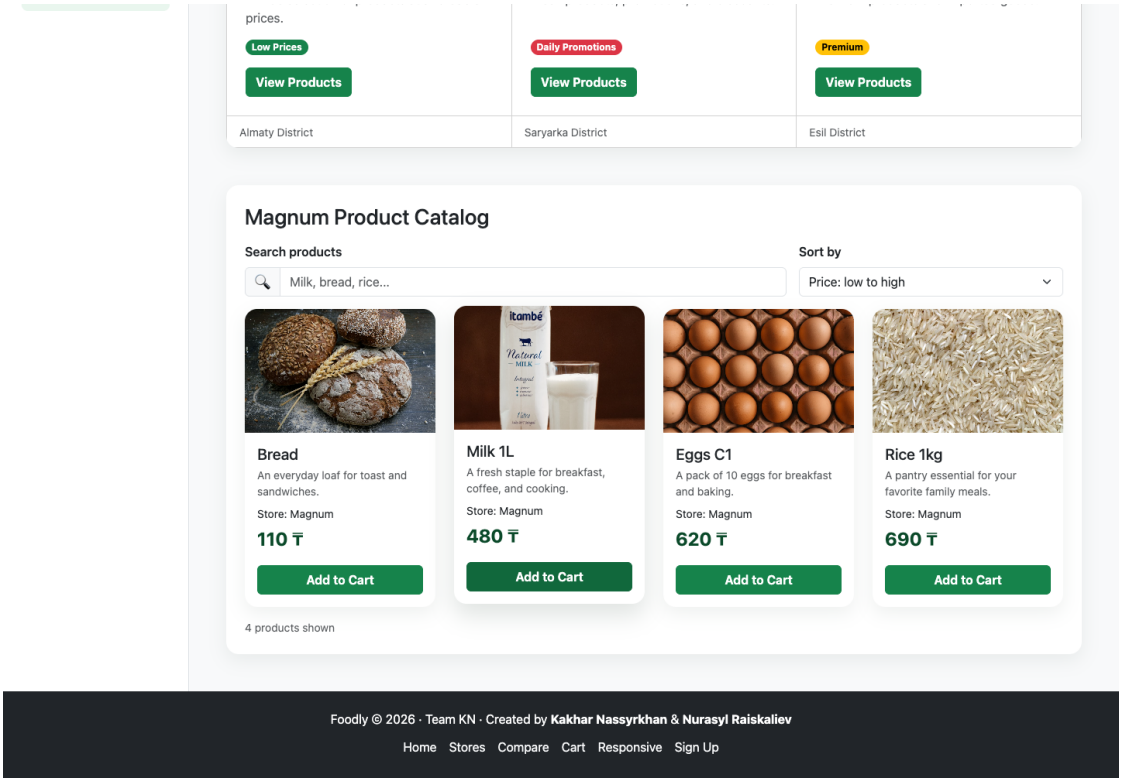


Figure 18. A real Add to Cart action produces a visible toast. This still image records the result; browser checks verify the animation property.

Existing FAQ expansion also uses a CSS transition, and the custom subscription popup has an entry animation. Motion is brief and attached to an explicit action.

9. Validation corrections

The four forms now share one validator. It checks required/trimmed values, practical email syntax, exactly ten national phone digits, name characters, password length and equality, address/message length and consent. Input bounds are checked in HTML and JavaScript.

js/validation.js · lines 41-61

```
41  function validateField(field) {
42      field.setCustomValidity("");
43      const value = field.value;
44      let message = "";
45      if (field.required && ((field.type === "checkbox" && !field.checked) ||
46          (field.type !== "checkbox" && !value.trim()))) {
47          message = field.type === "checkbox" ? "Please confirm this option." : "This field is
    required.";
48      } else if (value.trim() || field.required) {
49          message = rules[field.dataset.validate]?.(value, field) || "";
50      }
51      if (!message && field.maxLength > 0 && value.length > field.maxLength) {
52          message = `Use no more than ${field.maxLength} characters.`;
53      }
54      if (!message && !field.validity.valid) message = "Please choose or enter a valid
    value.";
55      field.setCustomValidity(message);
56      field.classList.toggle("is-invalid", Boolean(message));
57      field.setAttribute("aria-invalid", String(Boolean(message)));
58      document.getElementById(`${field.id}-error`).textContent = message;
59      touched.add(field);
60      return !message;
61  }
```

Figure 19. Actual source: js/validation.js, error computation, custom validity and accessible field state.

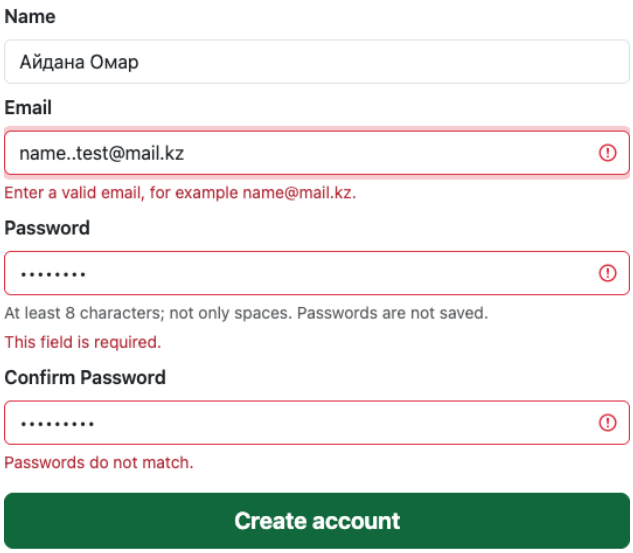


Figure 20. Registration rejects a malformed email, whitespace-only password and mismatching confirmation.

1. On submit, validate all relevant fields and focus the first error.

2. On subsequent input, recompute the error; changing the password also rechecks confirmation.

3. On success or Reset, clear validation state. Show an explicit demo result using textContent; never save passwords.

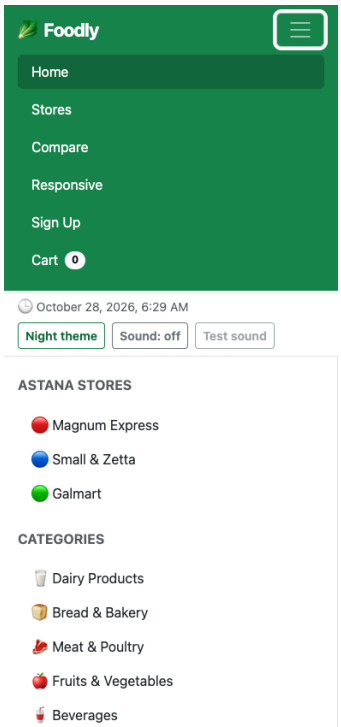
10. Structure and user experience

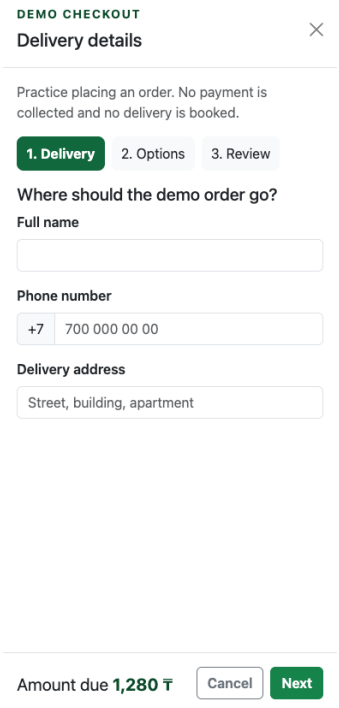
HTML defines content and controls; external CSS defines appearance; eight external application scripts define behavior. Shared validation avoids conflicting rules. Bootstrap 5.3.3 and images are local, so the submitted site does not depend on a CDN.

cart.html · lines 217-223

```
217  <script src="vendor/bootstrap.bundle.min.js" integrity="sha384-
     YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz" crossorigin="anonymous">
     </script>
218  <script src="js/validation.js" defer></script>
219  <script src="js/app.js" defer></script>
220  <script src="js/theme.js" defer></script>
221  <script src="js/datetime.js" defer></script>
222  <script src="js/checkout.js" defer></script>
223  <script src="js/interactions.js" defer></script>
```

Figure 21. Actual HTML source: external vendor and application scripts in cart.html.





Figures 22-23. Real 390 px screenshots: expanded mobile navigation and the checkout dialog.

Design and behavior choices

The existing grocery identity, six pages and store photos are retained. Shopping advice, ratings, sound and guided checkout extend the theme. Visible labels, focus outlines, inline errors, live messages and an optional sound toggle keep actions understandable.

No inline application scripts or event-handler attributes are used. Runtime style changes in JavaScript are limited to the explicit DOM-styling demonstration. Images retain alt text; responsive tables scroll inside their own containers.

11. Verification and regression tests

Check	Recorded outcome
Browser assertions	78 passed; Chrome 156.0.8078.49
Runtime / console / resources	0 observed errors on the main test page
HTML / JavaScript	6 HTML pages pass configured validation; 8 scripts pass syntax checks
Accessibility	0 axe violations on 6 default pages and dark homepage
Responsive widths	390, 768 and 1440 px; no page-level horizontal overflow
Audio lifecycle	2 starts, 2 ends; maximum 1 active oscillator

tests/browser.cjs · lines 167-180

```
167  await page.click('#checkout-next');
168  check(await page.locator('[data-checkout-step="1"]').isVisible(), 'Valid delivery proceeds
     to options');
169  await page.check('#pay-cash');
170  await page.fill('#order-note', '<img src=x onerror=alert(1)>');
171  await page.click('#checkout-back');
172  check(await page.inputValue('#customer-name') === 'Test Student', 'Back preserves delivery
     input');
173  await page.click('#checkout-next');
174  check(await page.locator('#pay-cash').isChecked(), 'Back/Next preserves payment choice');
175  await shot('checkout-options', '.modal-content');
176  await page.click('#checkout-next');
177  check((await page.textContent('#checkout-review')).includes('Cash on delivery'), 'Object
     method maps payment choice');
178  check(await page.locator('#checkout-review img').count() === 0, 'Review renders user input
     as text, not HTML');
179  await page.click('#checkout-submit');
180  check(await page.locator('#checkout-terms').getAttribute('aria-invalid') === 'true', 'Final
     consent is required');
```

Figure 24. Actual regression-test source: wizard transitions, input preservation, review and consent checks.

What the test run proves

The suite covers invalid and successful forms, keyboard flow, persistence, sorting, cross-tab cart synchronization, corrupt/blocked storage, safe review text, reduced motion, images and mobile checkout. Screenshots come from those real states.

The detailed assertion list and audit JSON files are in docs/. Automated accessibility checks cover the recorded states and do not replace manual assistive-technology testing. There is no real backend, payment or delivery integration.

12. Commit, push and deployment

1. Run the browser suite, HTML validation and JavaScript syntax checks.

2. Commit the implementation to the shared repository and push main. GitHub Pages builds from the repository root.

3. Wait for the Pages workflow to succeed, then open the live site and verify the new interactions and deployed source.

Implementation commit: df5c53d7dfdbb415e4f88307681fdbc601529f21
Pages result: Built successfully; live URL checked on 28 October 2026.
Live source check: All 16 HTML/CSS/JS files match the tested local source; live Read More and rating checks pass.

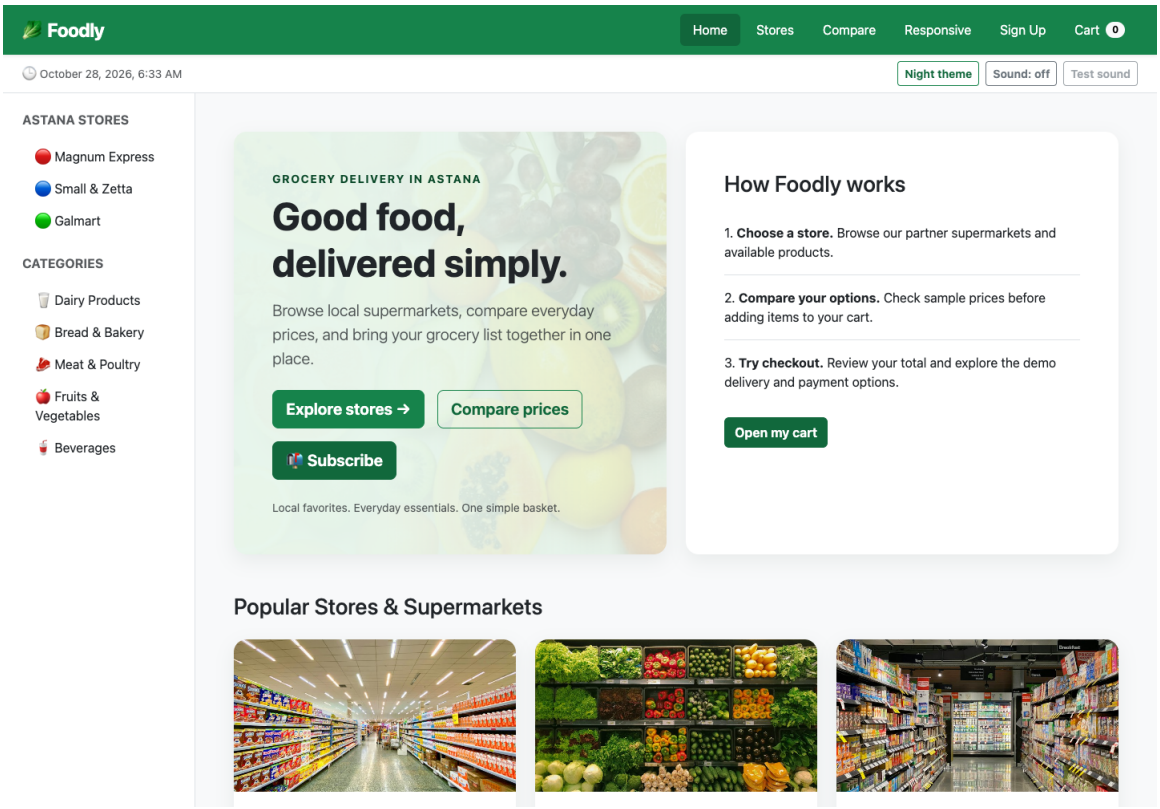


Figure 25. The updated site loaded from the public GitHub Pages URL after successful deployment.

Live website
https://vuus1d.github.io/Food-Delivery-Platform-KN/
Deployment workflow
https://github.com/vuus1d/Food-Delivery-Platform-KN/actions/runs/40615536747

This report records the tested implementation commit. A subsequent documentation commit adds this PDF and deployment evidence; it does not change the application implementation. The ZIP includes the project and report.

Reflection and defense preparation

Challenges and resolutions

Inconsistent validation. Separate form rules previously behaved differently. A shared validator now owns the rules, error rendering, reset handling and callbacks.

Moving controls during input. Displaying a new error on blur could move a checkbox during a click. Validation now starts on submission and rechecks fields already inspected when their values change.

Preserving wizard state. Rebuilding inputs would discard user values. The wizard keeps sections in the DOM and toggles visibility; final submission checks every step again.

Sound overlap. Rapid clicks could create overlapping tones. A busy guard is set before asynchronous resume, and ended oscillators disconnect and release it.

Reliable presentation. The night palette needed stronger secondary-button contrast. Local Bootstrap assets remove CDN dependence for classroom demonstration.

What to explain at the defense

Show how DOM selectors find controls, how textContent differs from HTML insertion, why preventDefault is needed, how callbacks update the checkout step, and how switch, map, filter, reduce and for...of serve different purposes. Demonstrate invalid input, Back/Next preservation, theme persistence and non-overlapping sound.

Use docs/DEFENSE-RU.md for a short demonstration sequence and source-based answers. Both team members should be able to explain and modify the code themselves; a completed project does not replace the practice defense.

Submission checklist

Submit the PDF report and project ZIP separately in LMS. Every team member must submit. Check the course deadline in lms.astanait.edu.kz and attend the practice lesson defense. The LMS deadline and submission status were not available in the supplied project and are not claimed as completed.

Source of requirements

Frontend Assignment 5 JS and DOM Manipulation.pdf, supplied with the task: Part 1 (Advanced JS), Part 2 (Deploy the Updated Website), submission rules and evaluation criteria. Project code and screenshots are included in the accompanying ZIP.